Section 1: WebSocket + STOMP Protocol

Ridesharing Backend — Learning Notes

---

1. HTTP vs WebSocket

HTTP is a **request-response** protocol — the client always initiates. The server can never reach out unprompted. In a ridesharing app this means a passenger couldn't instantly know when a driver accepts — they'd have to keep asking (*polling*), which is slow and wasteful.

WebSocket solves this with a **persistent, bidirectional connection**. After a one-time HTTP upgrade handshake, both sides can send messages freely at any time.

HTTP	WebSocket
Request-response only	Bidirectional, server can push
Connection closes after reply	Connection stays open
Client must poll for updates	Server pushes instantly
Good for: REST APIs, page loads	Good for: live location, ride status

2. Why STOMP on top of WebSocket?

Raw WebSocket is just a pipe — you can send strings or bytes, but there is no structure. STOMP (Simple Text Oriented Messaging Protocol) adds a thin layer that defines **frames**:

- **Command** — SEND, SUBSCRIBE, MESSAGE, CONNECT…
- **Headers** — destination, content-type, etc.
- **Body** — your JSON payload

Example STOMP frame sent from server to passenger:

```
MESSAGE
destination:/topic/passenger/42/ride-updates
content-type:application/json

{"driverId":7,"driverName":"Ravi Kumar","status":"MATCHED","eta":"5 min"}
```

Example SUBSCRIBE frame sent by passenger app on connect:

```
SUBSCRIBE
destination:/topic/passenger/42/ride-updates
```

3. Spring WebSocket Configuration

The WebSocket dependency enables support; your config class shapes how it works:

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
```

```
    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        // In-memory broker handles /topic destinations
        config.enableSimpleBroker("/topic");
        // Client messages to /app/... route to @MessageMapping methods
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // HTTP upgrade URL, with SockJS fallback
        registry.addEndpoint("/ws").withSockJS();
    }
}
```

Method	Purpose
registerStompEndpoints	Defines the /ws handshake URL (one-time HTTP upgrade)
configureMessageBroker	Sets up in-memory broker + prefixes
enableSimpleBroker	Creates in-memory broker for /topic destinations
setApplicationDestinationPrefixes	Routes /app/... to @MessageMapping methods
withSockJS()	Adds fallback for browsers without WebSocket

4. /app vs /topic — the key mental model

There are **three distinct jobs** in the STOMP setup:

Concept	Direction / Role
/ws endpoint	One-time HTTP → WebSocket upgrade handshake
@MessageMapping	Handles messages FROM client TO server
SimpMessagingTemplate	Pushes messages FROM server TO client
/app prefix	Client sending to server (routes to @MessageMapping)
/topic prefix	Server sending to client (broker manages subscribers)

Key insight

- @MessageMapping is like @RequestMapping — it handles incoming client messages.
- SimpMessagingTemplate has no HTTP equivalent — HTTP simply cannot push.
- The broker is a post office: it holds the subscriber list and delivers to matching sessions.
- SimpMessagingTemplate is a client OF the broker — it hands messages off, doesn't manage subscribers.

5. @MessageMapping and @SendTo

```
@Controller
public class RideWebSocketController {

    // Client sends to /app/ride.update
    @MessageMapping("/ride.update")
    @SendTo("/topic/rides")  // broadcast to ALL subscribers
    public RideStatusUpdate handleRideUpdate(RideUpdateRequest req) {
        return new RideStatusUpdate(req.getRideId(), "MATCHED");
    }
}
```

@SendTo is a shortcut for broadcasting the return value. For targeted messages (one specific user), use SimpMessagingTemplate directly.

6. SimpMessagingTemplate — server-initiated push

This is the most important tool for ridesharing. The trigger comes from a REST API call, not a WebSocket message — the server needs to push proactively.

```
@Service
@RequiredArgsConstructor
public class RideNotificationService {

    private final SimpMessagingTemplate messagingTemplate;

    public void notifyPassengerRideAccepted(Long passengerId,
                                            Long driverId, String eta) {
        Map<String, Object> payload = Map.of(
            "driverId", driverId,
            "status",   "MATCHED",
            "eta",      eta
        );
        // Hands message to broker → broker delivers to subscriber
        messagingTemplate.convertAndSend(
            "/topic/passenger/" + passengerId + "/ride-updates",
            payload
        );
    }
}
```

7. Personal topics vs broadcast topics

Destination	Who receives it
/topic/surge-pricing	All connected clients (all drivers)
/topic/passenger/42/ride-updates	Only passenger 42
/topic/driver/7/new-offer	Only driver 7
/topic/driver/7/ride-cancelled	Only driver 7

Security note

- The in-memory broker does NOT enforce who can subscribe to what.
- Any client that knows the destination string can subscribe to it.
- In production: add Spring Security WebSocket support to restrict subscriptions.

8. Full flow: driver accepts → passenger notified

Note: the trigger is a REST call, not a WebSocket message.

Step	Who	What happens
1	Driver app	POST /api/offers/{id}/accept (HTTP REST)
2	REST Controller	@PostMapping validates and calls RideService
3	RideService	Updates DB status to MATCHED, calls NotificationService
4	NotificationService	SimpMessagingTemplate.convertAndSend(...)
5	In-memory broker	Looks up subscriber for /topic/passenger/42/ride-updates
6	Passenger app	Receives STOMP MESSAGE frame, shows driver-found screen

9. Client-side JavaScript (reference)

```
const socket = new SockJS('/ws');
const stompClient = Stomp.over(socket);

stompClient.connect({}, () => {
    stompClient.subscribe(
        `/topic/passenger/${passengerId}/ride-updates`,
        (message) => {
            const update = JSON.parse(message.body);
            if (update.status === 'MATCHED') {
                showDriverFoundScreen(update);
            }
        }
    );
});
```

Section 1 — what you learned

- HTTP is request-response only; WebSocket is persistent and bidirectional.
- STOMP adds structured frames (command + headers + body) over raw WebSocket.
- SockJS provides a fallback for browsers without WebSocket support.
- /app prefix routes client messages to @MessageMapping methods.
- /topic prefix is managed by the in-memory broker for server-to-client delivery.
- SimpMessagingTemplate is a client OF the broker — it hands off messages, doesn't manage subs.
- Personal topics (/topic/passenger/42/...) target one user; broadcast topics target all.
- Server-initiated pushes (e.g. driver accepted) come via SimpMessagingTemplate, not @MessageMapping.

Next: Section 2 — Fare Calculation + Haversine Formula